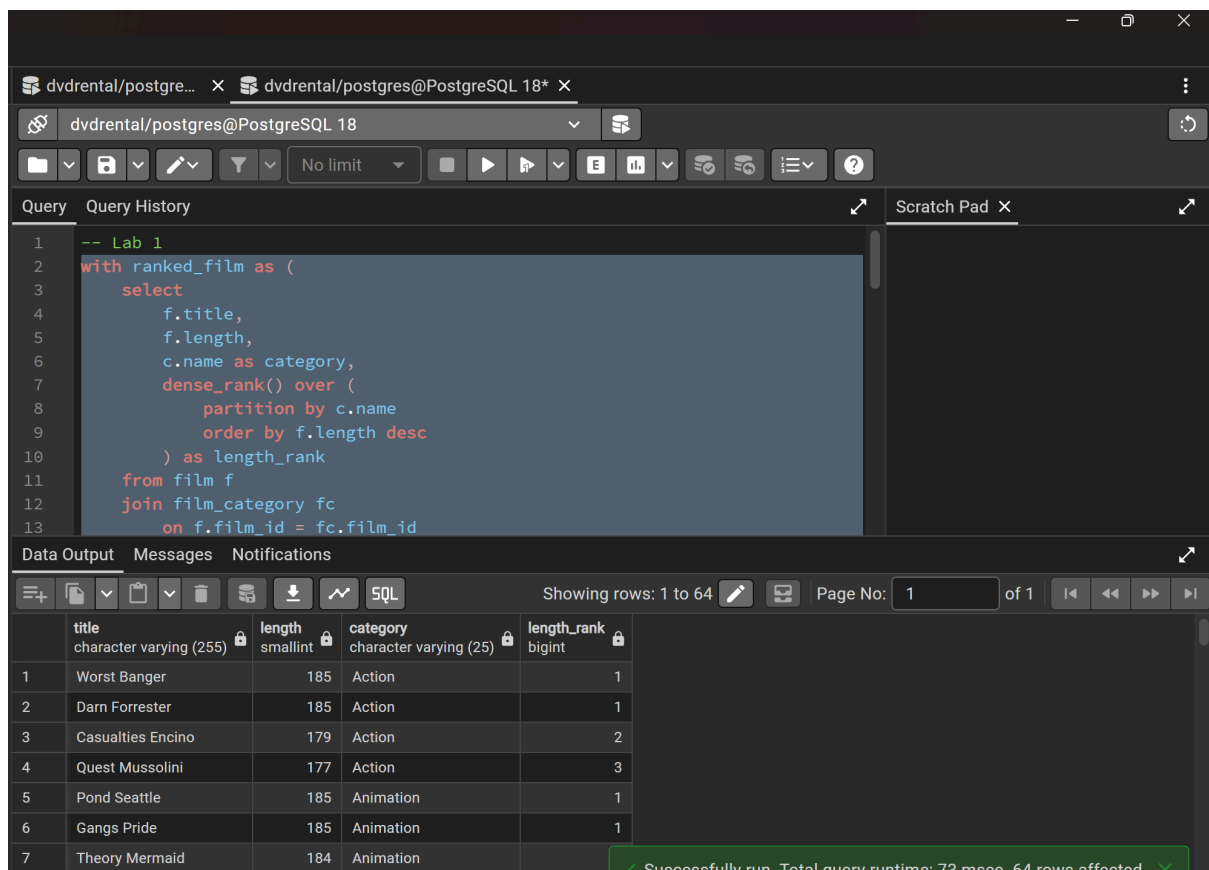


Lab1

```
with ranked_film as (  
    select  
        f.title,  
        f.length,  
        c.name as category,  
        dense_rank() over (  
            partition by c.name  
            order by f.length desc  
        ) as length_rank  
    from film f  
    join film_category fc  
        on f.film_id = fc.film_id  
    join category c  
        on fc.category_id = c.category_id  
)  
select  
    title,  
    length,  
    category,  
    length_rank  
from ranked_film  
where length_rank <= 3  
order by category,length_rank;
```



The screenshot shows a PostgreSQL IDE interface. The top panel displays the query being executed. The bottom panel shows the results of the query, which are 7 rows of data. The results are displayed in a table with columns: title, length, category, and length_rank. The data is sorted by category and then by length_rank.

	title	length	category	length_rank
1	Worst Banger	185	Action	1
2	Darn Forrester	185	Action	1
3	Casualties Encino	179	Action	2
4	Quest Mussolini	177	Action	3
5	Pond Seattle	185	Animation	1
6	Gangs Pride	185	Animation	1
7	Theory Mermaid	184	Animation	

Successfully run. Total query runtime: 73 msec. 64 rows affected.

Lab2

```
with monthly_revenue as (  
    select  
        DATE_TRUNC('month',payment_date) as month,  
        sum(amount) as revenue  
    from payment  
    group by DATE_TRUNC('month',payment_date)  
)  
select  
    month,  
    revenue,  
    LAG(revenue) over(  
        order by month  
    ) as previous_revenue  
from monthly_revenue  
order by month;
```

The screenshot shows a PostgreSQL IDE interface. The top bar displays the database name 'dvdrental/postgres' and the connection name 'PostgreSQL 18'. The main query editor contains the following SQL code:

```
31 from payment  
32 group by DATE_TRUNC('month',payment_date)  
33 )  
34 select  
35     month,  
36     revenue,  
37     LAG(revenue) over(  
38         order by month  
39     ) as previous_revenue  
40 from monthly_revenue  
41 order by month;  
42  
43 -- lab3  
44 select
```

The bottom panel shows the 'Data Output' tab with the following results:

	month timestamp without time zone	revenue numeric	previous_revenue numeric
1	2007-02-01 00:00:00	8351.84	[null]
2	2007-03-01 00:00:00	23886.56	8351.84
3	2007-04-01 00:00:00	28559.46	23886.56
4	2007-05-01 00:00:00	514.18	28559.46

Lab3

```
select  
    payment_id,  
    customer_id,  
    payment_date,
```

```

        amount,
        sum(amount) over(
            partition by customer_id
            order by payment_date
        ) as running_total
    from payment
    order by customer_id, payment_date;

```

The screenshot shows a PostgreSQL query editor interface. The query is as follows:

```

47     payment_date,
48     amount,
49     sum(amount) over(
50         partition by customer_id
51         order by payment_date
52     ) as running_total
53 from payment
54 order by customer_id, payment_date;
55
56 -- Lab4
57
58
59

```

The results are displayed in a table with the following columns: payment_id [PK] integer, customer_id smallint, payment_date timestamp without time zone, amount numeric (5,2), and running_total numeric. The table shows 7 rows of data.

payment_id [PK] integer	customer_id smallint	payment_date timestamp without time zone	amount numeric (5,2)	running_total numeric
1	18495	2007-02-14 23:22:38.996577	5.99	5.99
2	18496	2007-02-15 16:31:19.996577	0.99	6.98
3	18497	2007-02-15 19:37:12.996577	9.99	16.97
4	18498	2007-02-16 13:47:23.996577	4.99	21.96
5	18499	2007-02-18 07:10:14.996577	4.99	26.95
6	18500	2007-02-18 12:02:25.996577	0.99	27.94
7	18501	2007-02-21 04:53:11.996577		

Successfully run. Total query runtime: 199 msec. 14596 rows affected.

Lab4

```

with customer_spending as (
    select
        c.customer_id,
        c.first_name,
        c.last_name,
        co.country,
        sum(p.amount) as total_spent
    from customer c
    join address a
        on c.address_id = a.address_id
    join city ci
        on a.city_id = ci.city_id
    join country co
        on ci.country_id = co.country_id
    join payment p

```

```

        on c.customer_id = p.customer_id
group by
    c.customer_id,
    c.first_name,
    c.last_name,
    co.country
),
ranked_customer as (
    select
        customer_id,
        first_name,
        last_name,
        country,
        total_spent,
        rank() over(
            partition by country
            order by total_spent desc
        ) as spending_rank
    from customer_spending
)
select
    customer_id,
    first_name,
    last_name,
    country,
    total_spent
from ranked_customer
where spending_rank = 1
order by country;

```

